

15

Graph Theory and Network

Undirected Graph, Directed Graph, Adjacency Matrix, Laplacian Matrix

Graph theory is the branch of mathematics that studies graph structures made up of nodes and edges. Graphs and networks are everywhere — in machine learning, artificial intelligence, and even everyday life. Matrices and matrix decompositions are excellent tools for describing and analyzing graphs and networks.

This chapter serves as an introductory tutorial on graph theory and networks — and the various linear algebra tools introduced earlier in this book show up everywhere!

15.1 Undirected Graph



The core skills you'll master in this section:

- ▶ In an undirected graph, edges have no direction — connections are symmetric.
- ▶ The adjacency matrix of an undirected graph is symmetric.
- ▶ Powers of the adjacency matrix give the number of paths between nodes that pass through multiple intermediate nodes.
- ▶ Summing the adjacency matrix along its rows or columns computes each node's degree.
- ▶ A weighted undirected graph has edges that carry weights.

This section introduces undirected graphs; the next section covers directed graphs. We recommend reading the two sections side by side.

Undirected Graph

An undirected graph is a graph whose edges have no direction — an edge simply represents a connection between two nodes. In other words, if an edge exists between two nodes, it means they're connected to each other, and the connection is symmetric.

Take, for example, Figure 1 — the park has 4 attractions in total, each of which can be regarded as a node. In Figure 1, this graph has four nodes in total: a, b, c, and d.

If a hiking trail exists between two attractions, the corresponding nodes are connected with a line segment, and that line segment is an edge. For example, if a hiking trail exists between nodes a and b,

it corresponds to edge ab . Clearly, edge ab has no direction, since we can walk from a to b , or conversely from b to a . This is exactly why Figure 1 is called an undirected graph.

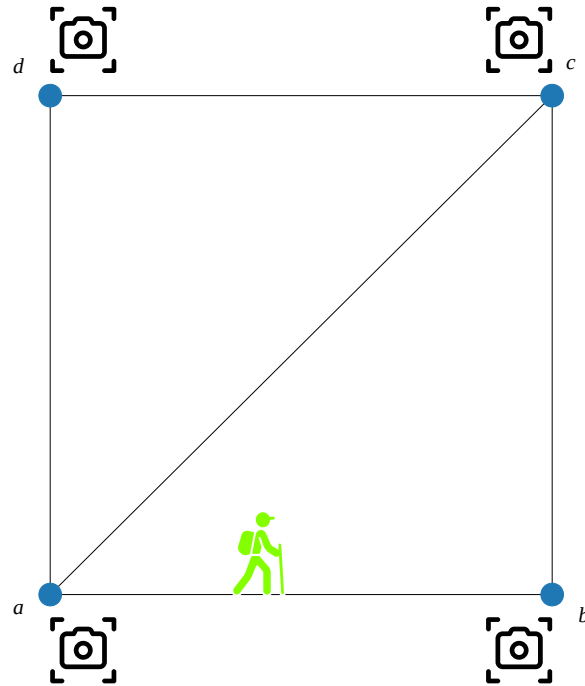


Figure 1. The Undirected Graph Formed by the Park's Attractions

Drawing an Undirected Graph with NetworkX

Code 1 uses NetworkX to draw Figure 1 – this undirected graph. Let's walk through the key statements below.

a Import a library specifically designed for working with graph structures, called NetworkX. It supports the creation, manipulation, and analysis of various types of graphs (such as undirected graphs, directed graphs, weighted graphs, and more). Here we give it the abbreviation `nx`; all subsequent NetworkX functions are called via `nx.xxx()`.

b Create an empty undirected graph object. An undirected graph is one where none of the edges have a direction – for example, "a and b are connected" means the two can reach each other, with no order implied. This graph is named `undirected_G`; you can think of it as a container, into which we'll add nodes and edges next.

c Use `add_nodes_from(['a','b','c','d'])` to add four nodes: 'a', 'b', 'c', and 'd'. These nodes are the most basic elements in a graph – like users in a social network, locations on a map, or web pages on the internet. The function `add_nodes_from()` can add multiple nodes at once; just pass them in as a list.

d Use `add_edges_from([(a,b), (a,c), (a,d), (b,c), (c,d)])` to add edges to the undirected graph. Each edge connects two nodes – for example, `(a,b)` means there's an edge between a and b. This function can also add multiple edges at once, by passing in a list of edges. Note that, since we created an undirected graph earlier, `(a,b)` and `(b,a)` are the same.

e Assign each node in the graph a position in the 2D plane. This creates a dictionary whose keys are node names and whose values are their position coordinates. For example, 'a' is at position `[0, 0]`,

meaning it's placed at the bottom-left of the graph. The purpose of this step is to make the graph look clearer and more organized when drawn, with positions controlled manually by us.

f Use `nx.draw_networkx()` to draw the undirected graph defined by the earlier code.

The parameter `undirected_G` is the graph object we want to draw.

`pos = pos` tells it where each node should be placed (i.e., the coordinates defined earlier).

`node_size = 188` controls the size of each node's circle – the larger the number, the bigger the node.

By default, the function automatically draws the nodes and edges, and labels each node with its name.

Code 1. Drawing an Undirected Graph with NetworkX |  LA_15_01_01.ipynb

```
## 初始化
import matplotlib.pyplot as plt
a import networkx as nx

## 无向图
b undirected_G = nx.Graph()

## 添加节点
c undirected_G.add_nodes_from(['a', 'b', 'c', 'd'])

## 添加边
d undirected_G.add_edges_from([('a', 'b'),
                                ('a', 'c'),
                                ('a', 'd'),
                                ('b', 'c'),
                                ('c', 'd')])

## 定义节点坐标
e pos = {'a': [0, 0],
          'b': [1, 0],
          'c': [1, 1],
          'd': [0, 1]}

## 可视化
f plt.figure(figsize = (6,6))
    nx.draw_networkx(undirected_G, pos = pos, node_size = 188)
```

Adjacency Matrix

Figure 1 – this undirected graph can be represented using an adjacency matrix.

In simple terms, the adjacency matrix of an undirected graph is a matrix form used to represent the connection relationships between nodes in the graph. For an undirected graph with n nodes, its adjacency matrix is an $n \times n$ symmetric matrix, denoted A .

For example, Figure 1's adjacency matrix, as an undirected graph, is

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

where the element a_{ij} in row i , column j indicates whether an edge exists between node i and node j .

As shown in Figure 2 (a), an undirected edge exists between nodes a and b in the undirected graph, so the element in row 1, column 2 of matrix $\mathbf{A}$ is 1; and since undirected edges have no direction, the element in row 2, column 1 of matrix $\mathbf{A}$ is also 1.

As shown in Figure 2 (f), no undirected edge exists between nodes c and d in the undirected graph, so the element in row 3, column 4 of matrix $\mathbf{A}$ is 0; and the element in row 4, column 3 of matrix $\mathbf{A}$ is also 0.

You may have already noticed that, in the equation above, matrix $\mathbf{A}$'s main-diagonal elements are all 0. This is because, as shown in Figure 1, no node has an edge to itself – such an edge, if it existed, would be called a self-loop.

In simple terms, a self-loop is an edge that connects a node to itself. For example, if attraction a has a self-loop, it means there's a hiking trail that starts at a and returns to a .

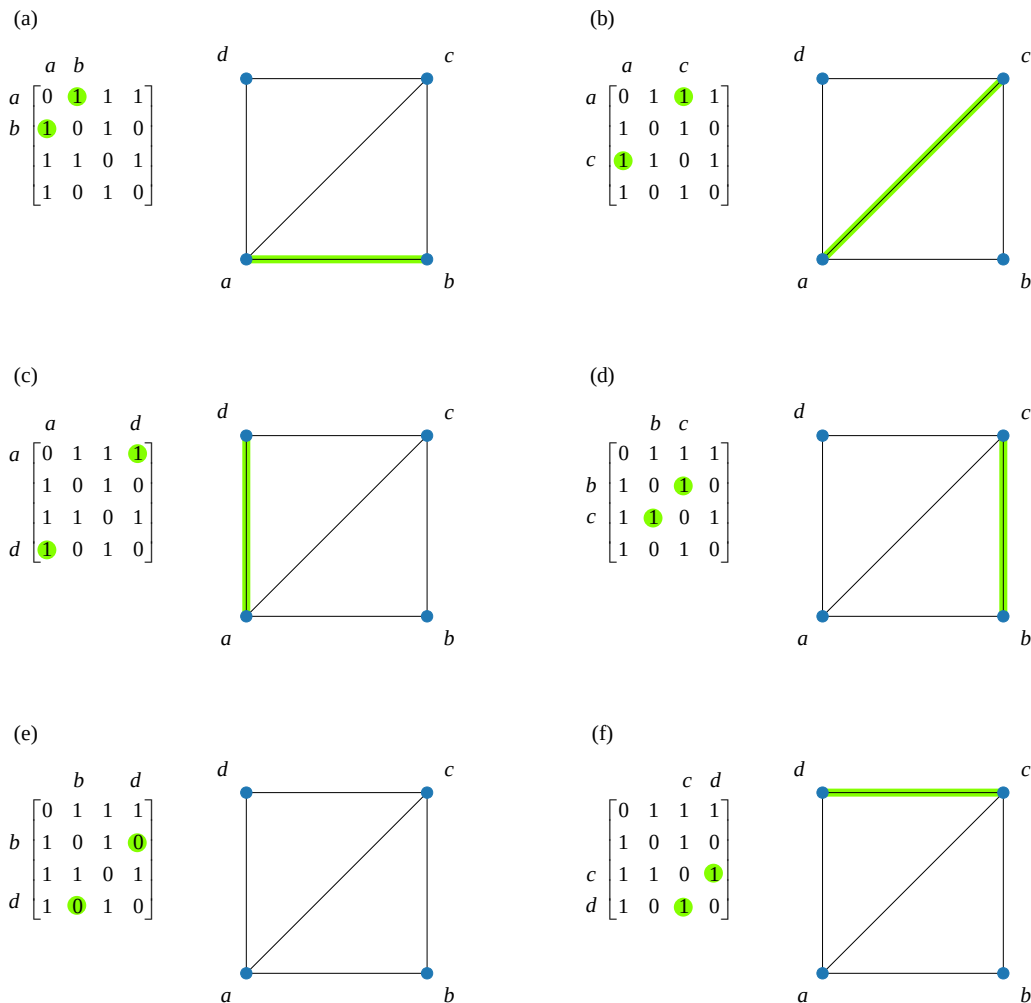


Figure 2. Each Element of the Undirected Graph's Adjacency Matrix

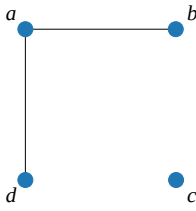
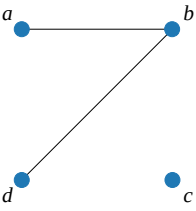


LA_15_01_01.ipynb also uses `networkx.adjacency_matrix()` to compute the undirected graph's adjacency matrix.

? Please analyze each undirected graph and adjacency matrix in Table 1 one by one.

Table 1. Several Undirected Graphs and Adjacency Matrices Constructed From 4 Nodes (table example from The Way of Data)

Undirected Graph	Adjacency Matrix	Undirected Graph	Adjacency Matrix
	$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$
	$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$
	$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$
	$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$
	$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$

	$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$
---	--	--	--

From Adjacency Matrix to Undirected Graph

Code 2 shows how to construct an undirected graph from an adjacency matrix (a 2D array). Let's walk through the key statements below.

- a** Use `numpy.array()` to define a 4×4 adjacency matrix representing a graph's structure. Since the graph is undirected, the adjacency matrix is symmetric.
- b** Call NetworkX's `Graph` function to create an undirected graph from the adjacency matrix. This function automatically adds an edge wherever the matrix has a 1. `nodetype=int` means the node labels are integers, such as 0, 1, 2, 3. Once created, the graph is stored in the variable `undirected_G`.
- c** Construct a dictionary that renames the original node labels (0, 1, 2, 3) to letters (a, b, c, d). This makes the graph easier to read and makes it more convenient to label nodes in the graph.
- d** Use NetworkX's `relabel_nodes` function to rename the graph's node labels according to the mapping dictionary above. After this, the nodes in the graph are no longer numbers but letters — for example, node 0 becomes node 'a'.

The remaining code was already covered earlier in this section; please study it line by line by annotating each statement.

Code 2. From Adjacency Matrix to Undirected Graph |  LA_15_01_02.ipynb

```

## 初始化
import matplotlib.pyplot as plt
import networkx as nx
import numpy as np

## 定义邻接矩阵
a adjacency_matrix = np.array([[0, 1, 1, 1],
                               [1, 0, 1, 0],
                               [1, 1, 0, 1],
                               [1, 0, 1, 0]])

## 创建无向图
# 用邻接矩阵创建无向图
b undirected_G = nx.Graph(adjacency_matrix, nodetype=int)

# 构造节点映射
c mapping = {0: 'a', 1: 'b', 2: 'c', 3: 'd'}

# 重命名节点
d undirected_G = nx.relabel_nodes(undirected_G, mapping)

## 定义节点坐标
pos = {'a': [0, 0],
       'b': [1, 0],
       'c': [1, 1],
       'd': [0, 1]}

## 可视化
plt.figure(figsize = (6,6))
nx.draw_networkx(undirected_G, pos = pos,
                  node_size = 188, with_labels=True)

```

Looking Again at the Adjacency Matrix

As shown in Figure 3, in row 1 of adjacency matrix A (aside from the main-diagonal element), the elements equal to 1 represent the nodes that a can reach directly. In other words, without passing through any other node, node a can walk directly to b, c, and d.

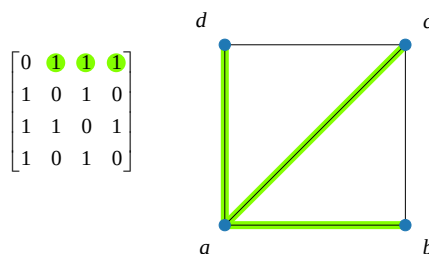


Figure 3. a Directly Reaches Three Nodes — Row 1 of Adjacency Matrix A

As shown in Figure 4, column 1 of adjacency matrix A gives the same information.

? Please use the same approach to analyze the remaining rows and columns of adjacency matrix A .

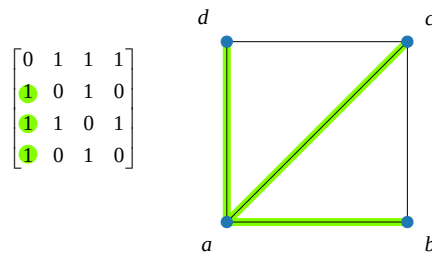


Figure 4. a Directly Reaches Three Nodes — Column 1 of Adjacency Matrix A

The number of paths from node a to node c passing through exactly one intermediate node can be obtained via the following matrix multiplication

$$A @ A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} @ \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 1 & 2 & 1 \\ 1 & 2 & 1 & 2 \\ 2 & 1 & 3 & 1 \\ 1 & 2 & 1 & 2 \end{bmatrix}$$

As shown in Figure 5, the element in row 1, column 3 (or row 3, column 1) of the result of $A @ A$ is exactly the number of paths from node a to node c passing through exactly one intermediate node.

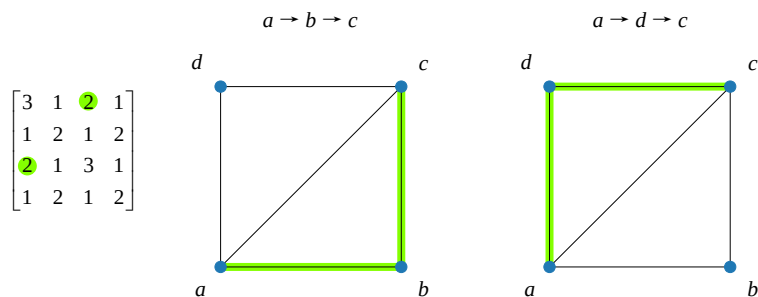


Figure 5. Paths From Node a to Node c Passing Through One Intermediate Node

The main-diagonal elements of $A @ A$, on the other hand, represent paths that start at a node and return to that same node. For example, as shown in Figure 6, there are 3 paths from node a, through one intermediate node, back to a.

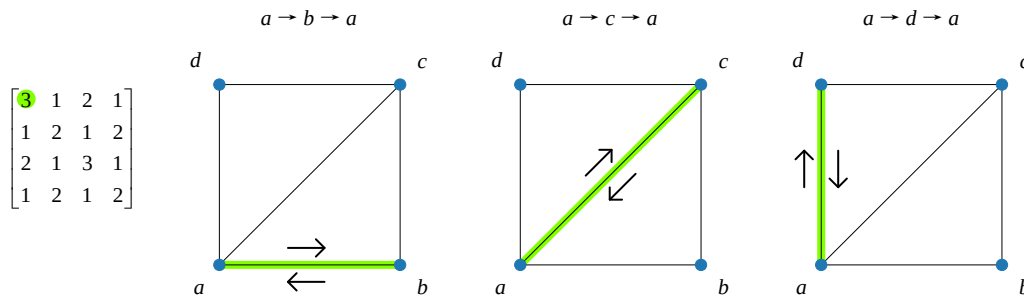


Figure 6. Paths From Node a, Through One Intermediate Node, Back to a

? Please analyze, in the equation above, the meaning of the remaining elements in the result of $A@A$.

To compute the number of paths from a to c that pass through at most one intermediate node, we can use the following operation

$$A + A@A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} + \begin{bmatrix} 3 & 1 & 2 & 1 \\ 1 & 2 & 1 & 2 \\ 2 & 1 & 3 & 1 \\ 1 & 2 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 3 & 2 & 3 & 2 \\ 2 & 2 & 2 & 2 \\ 3 & 2 & 3 & 2 \\ 2 & 2 & 2 & 2 \end{bmatrix}$$

Please analyze the result above on your own.

Degree and Degree Matrix

In an undirected graph, the degree of a node is the number of connections it has to other nodes in the graph. The degree reflects how active a node is.

Specifically, for an undirected graph, a node's "degree" is the number of edges connected to it. Each additional edge increases the node's degree by one. For example, in Figure 1, node a is connected by edges to b, c, and d, so a's degree is 3.

The degree matrix is a square matrix used to record the degrees of all nodes.

The degree matrix is a diagonal square matrix — that is, every element except those on the main diagonal is 0. Each value on the main diagonal corresponds to a node's degree.

For example, Figure 1's degree matrix, as an undirected graph, is

$$D = \begin{bmatrix} 3 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 2 \end{bmatrix}$$

As shown in Figure 7, this degree matrix tells us that the degrees of the four nodes a, b, c, and d are 3, 2, 3, and 2, respectively.

In a social network, a node with a high degree is like a "social butterfly"; a node with a low degree tends to stay away from socializing and prefers being alone.

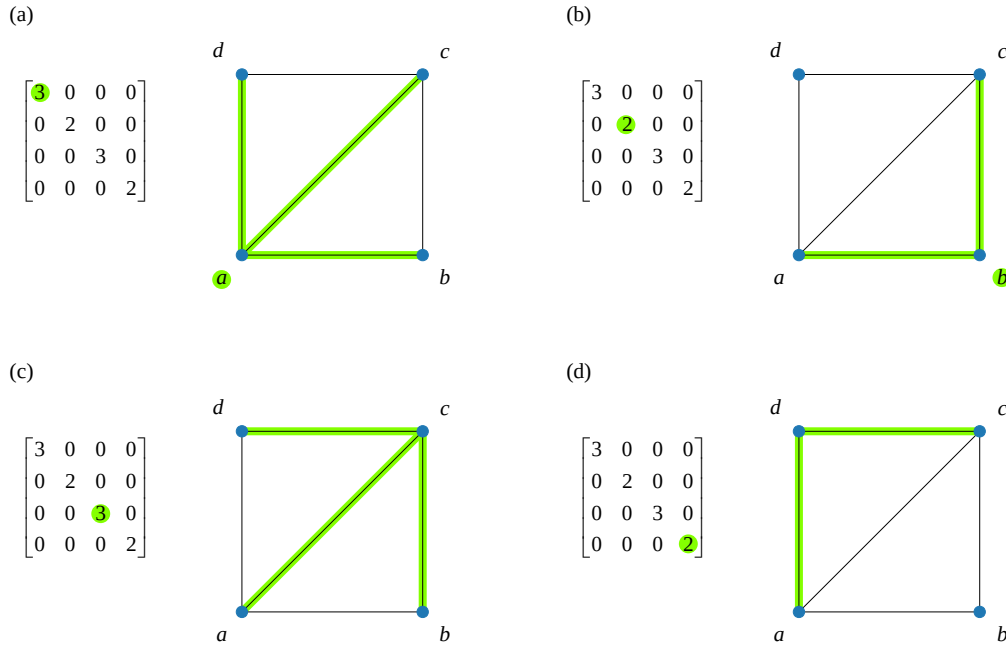


Figure 7. The Degree Matrix of an Undirected Graph

Summing the undirected graph's adjacency matrix A along its columns gives each node's degree. This can be done using the following matrix multiplication

$$\mathbf{1}^T A = \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 2 & 3 & 2 \end{bmatrix}$$

The resulting values are then passed to the `diag()` function to construct the diagonal square matrix.

Since the undirected graph's adjacency matrix A is symmetric, summing along its rows also gives each node's degree, that is

$$A \mathbf{1} = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \\ 3 \\ 2 \end{bmatrix}$$

Degree and the degree matrix are important building blocks for studying graph structure, graph computation, and graph algorithms; they're often used together with the adjacency matrix – for example, when computing a graph's Laplacian matrix.



LA_15_01_03.ipynb computes the degree matrix using several methods; please study it on your own.

Weighted Undirected Graph

A weighted undirected graph is one in which every edge has a numerical weight; by contrast, the undirected graph discussed earlier in this section is also called an unweighted undirected graph.

An undirected graph's edge weights can be used to represent many kinds of features. Taking Figure 1 from earlier in this section as an example, an edge's weight could be the distance between nodes, walking time, and so on.

For example, as shown in Figure 8, if we label the undirected graph with the average walking time between nodes, we get a weighted undirected graph.

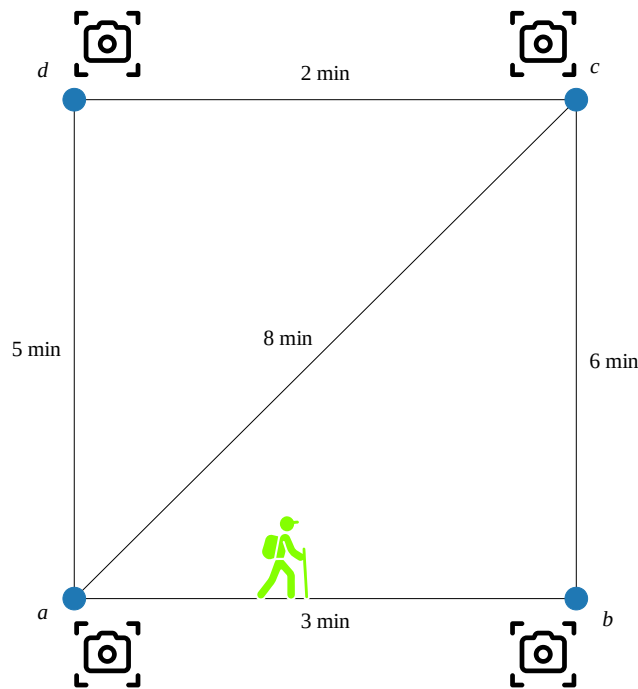


Figure 8. The Undirected Graph Formed by the Park's Attractions, with Weights Equal to the Average Walking Time Between Nodes (Minutes)



LA_15_01_04.ipynb shows how to construct and visualize a weighted undirected graph.



Please use tools such as DeepSeek/ChatGPT to complete the following exercises for this section.

Q1. Please use NetworkX to construct and draw each of the undirected graphs in Table 1 one by one.

Q2. Please compute the degrees of the nodes for each undirected graph in Table 1.

Q3. Please compute the adjacency matrices of all the undirected graphs in Table 1.

Q4. Please compute the degree matrices of all the undirected graphs in Table 1.

Q5. Please study the concepts of complete graph and empty graph on your own, and identify which graphs in Table 1 are the complete graph and which is the empty graph.